

Assignment — Modules & Environments

Topic

Python Modules | Packages | Third-Party Libraries | Virtual Environments

Scenario: Employee Management & Reporting System

Your organization is developing an **Employee Management System**.

The application has grown beyond a single Python file. The development team wants to organize the application using:

- Python Modules
- Python Packages
- Third-Party Libraries
- Virtual Environments
- Dependency Management

The organization has two different teams building two different applications using the same employee data.

Business Scenario

Project 1 – HR Report Generator

The HR team needs a report that can be generated in a **structured text/report format**.

The application must use:

- **Jinja2** → Generate the employee report from a template
 - **PrettyTable** → Display employee information as a formatted table
-

Project 2 – Employee CLI Application

The Operations team needs a **command-line application** for viewing employee information.

The application must use:

- **Tabulate** → Display employee information in a CLI table
 - **Rich** → Display employee information using a Rich terminal table
-

Important Requirement

These are two independent projects.

Each project must have its **own virtual environment**.

```
hr_report_generator/  
└─ .venv/  
  
employee_cli/  
└─ .venv/
```

Demonstrate that installing a library in one project's virtual environment does not make it available in the other project's environment.

Part 1 – Create Python Modules

Create the following modules:

```
employee.py  
salary.py  
attendance.py
```

employee.py

Implement:

```
add_employee()  
get_employee()  
get_all_employees()
```

Use simple Python data structures.

Example:

```
employees = [  
    {  
        "id": "E001",  
        "name": "John",  
        "department": "IT",  
        "salary": 50000  
    },  
    {  
        "id": "E002",  
        "name": "Alice",  
        "department": "HR",  
        "salary": 45000  
    },  
    {  
        "id": "E003",  
        "name": "Bob",  
        "department": "Finance",  
        "salary": 55000  
    }  
]
```

No database is required.

salary.py

Implement:

```
calculate_salary()  
calculate_bonus()
```

attendance.py

Implement:

```
mark_attendance()  
get_attendance()  
calculate_attendance_percentage()
```

Part 2 – Create a Python Package

Organize the modules into an `employee_system` package.

```
employee_system/  
|  
├─ __init__.py  
├─ employee.py  
├─ salary.py  
└─ attendance.py
```

Your application should be able to import the modules like:

```
from employee_system.employee import get_all_employees  
from employee_system.salary import calculate_salary
```

should understand:

```
employee.py
```

is a **module**, while:

```
employee_system/
```

is a **package**.

PROJECT 1 – HR Report Generator

Scenario

The HR department wants an employee report that can be generated from a predefined template.

The report should contain:

1. Report title
2. Employee details
3. Department
4. Salary
5. A formatted employee table

The HR team has decided to use:

```
Jinja2      → Report/template generation  
PrettyTable → Employee table
```

Project 1 – Folder Structure

Create:

```
hr_report_generator/  
|  
├─ app.py  
├─ requirements.txt  
|  
├─ templates/  
│   └─ employee_report.txt  
└─ employee_system/  
    ├── __init__.py  
    ├── employee.py  
    ├── salary.py  
    └─ attendance.py
```

Create a virtual environment:

```
python -m venv .venv
```

Activate it.

Project 1 – Install Dependencies

Install:

```
pip install Jinja2  
pip install prettytable
```

Then generate:

```
pip freeze > requirements.txt
```

Project 1 – Jinja2 Requirement

Create:

```
templates/employee_report.txt
```

The template should contain employee information using Jinja2 syntax.

For example:

```
Employee Report
```

```
=====
```

```
Employee ID : {{ employee.id }}
```

```
Name       : {{ employee.name }}
```

```
Department : {{ employee.department }}
```

```
Salary     : {{ employee.salary }}
```

The Python application should load the template and provide employee data.

Project 1 – Expected Jinja2 Output

When the application is executed:

```
Employee Report
```

```
=====
```

```
Employee ID : E001
```

```
Name       : John
```

```
Department : IT
```

```
Salary     : 50000
```

For another employee:

```
Employee Report
```

```
=====
```

```
Employee ID : E002
```

```
Name       : Alice
```

```
Department : HR
```

```
Salary     : 45000
```

The important point is that the text is generated from the **Jinja2 template**, not hard-coded directly into the Python program.

Project 1 – PrettyTable Requirement

The HR team also wants all employees displayed as a table.

Use:

```
from prettytable import PrettyTable
```

Create the table dynamically from the employee data.

Project 1 – Expected PrettyTable Output

The output should look similar to:

```
+-----+-----+-----+-----+
| ID    | Name  | Department | Salary |
+-----+-----+-----+-----+
| E001  | John  | IT         | 50000  |
| E002  | Alice | HR         | 45000  |
| E003  | Bob   | Finance    | 55000  |
+-----+-----+-----+-----+
```

What should notice

PrettyTable produces an **ASCII box-style table** with:

```
+-----+-----+
|      |      |
+-----+-----+
```

Should not manually construct these lines.

The **PrettyTable library generates the formatting.**

Project 1 – Complete Sample Output

The final output of Project 1 could look like:

```
=====
HR EMPLOYEE REPORT
=====
```

```
Employee Report
=====
```

```
Employee ID : E001
Name        : John
Department  : IT
Salary      : 50000
```

```
Employee Report
=====
```

```
Employee ID : E002
Name        : Alice
Department  : HR
Salary      : 45000
```

```
Employee Table
=====
```

```
+-----+-----+-----+-----+
| ID    | Name  | Department | Salary |
+-----+-----+-----+-----+
| E001  | John  | IT         | 50000  |
| E002  | Alice | HR         | 45000  |
| E003  | Bob   | Finance    | 55000  |
+-----+-----+-----+-----+
```

PROJECT 2 – Employee CLI Application

Scenario

The Operations team does not need a report file.

They need an **interactive CLI application** where employee information can be displayed directly in the terminal.

The team has selected:

```
Tabulate → Simple CLI tables
Rich      → Richly formatted terminal tables
```

Project 2 – Folder Structure

Create:

```
employee_cli/  
|  
├─ app.py  
├─ requirements.txt  
└─ employee_system/  
    ├─ __init__.py  
    ├─ employee.py  
    ├─ salary.py  
    └─ attendance.py
```

Create a separate virtual environment:

```
python -m venv .venv
```

Do not use the `.venv` from Project 1.

Project 2 – Install Dependencies

Install:

```
pip install tabulate  
pip install rich
```

Then:

```
pip freeze > requirements.txt
```

Project 2 – Tabulate Requirement

Use:

```
from tabulate import tabulate
```

Display employee information using Tabulate.

For example:

```
print(
    tabulate(
        employees,
        headers="keys",
        tablefmt="grid"
    )
)
```

Project 2 – Expected Tabulate Output

Using the `grid` format, the output will look similar to:

```
+-----+-----+-----+-----+
| id   | name  | department | salary |
+=====+=====+=====+=====+
| E001 | John  | IT         | 50000  |
+-----+-----+-----+-----+
| E002 | Alice | HR         | 45000  |
+-----+-----+-----+-----+
| E003 | Bob   | Finance    | 55000  |
+-----+-----+-----+-----+
```

Should experiment with another format.

For example:

```
tablefmt="simple"
```

Expected output:

```
id      name      department      salary
----  -
E001   John      IT              50000
E002   Alice     HR              45000
E003   Bob       Finance         55000
```

What should notice

Unlike `PrettyTable`, `Tabulate` provides multiple predefined table formats.

Should experiment with at least **two formats**.

Project 2 – Rich Table Requirement

Now create another employee table using Rich.

Import:

```
from rich.console import Console
from rich.table import Table
```

Create a table with:

```
Employee ID
Name
Department
Salary
```

Add the employee data dynamically.

Should add at least:

- A table title
- Column styling
- Alignment

Project 2 – Expected Rich Output

The terminal output should look similar to:

Employee Details			
ID	Name	Department	Salary
E001	John	IT	50000
E002	Alice	HR	45000
E003	Bob	Finance	55000

Depending on the terminal, Rich may additionally display **colors and styling**.

Project 2 – Complete Sample Output

A possible CLI output:

```
=====
EMPLOYEE CLI APPLICATION
=====
```

Employee List - Tabulate

```
+-----+-----+-----+-----+
| id   | name  | department | salary |
+=====+=====+=====+=====+
| E001 | John  | IT         | 50000  |
+-----+-----+-----+-----+
| E002 | Alice | HR         | 45000  |
+-----+-----+-----+-----+
| E003 | Bob   | Finance    | 55000  |
+-----+-----+-----+-----+
```

Employee List - Rich

Employee Details

ID	Name	Department	Salary
E001	John	IT	50000
E002	Alice	HR	45000
E003	Bob	Finance	55000

Part 3 – Compare the Libraries

Must create a short comparison.

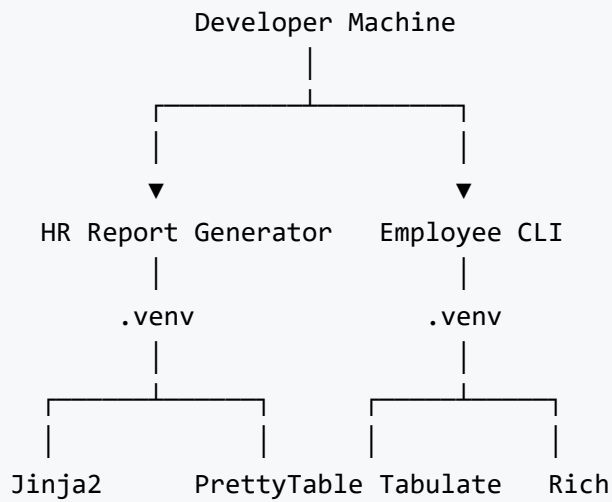
Library	Purpose	Output
Jinja2	Template-based text/report generation	Dynamic text
PrettyTable	Table formatting	ASCII box table
Tabulate	CLI table formatting	Multiple table styles
Rich	Rich terminal UI	Styled terminal table

Should explain why the company might choose one library over another.

Part 4 – Demonstrate Virtual Environment Isolation

This is the most important part of the assignment.

The two projects must have separate environments.



Test 1 – Project 1

Activate the HR Report Generator environment.

Run:

```
pip list
```

Should identify:

```
Jinja2
PrettyTable
```

Run the application.

The Jinja2 report and PrettyTable output should work.

Test 2 – Project 2

Deactivate Project 1's environment.

Activate Project 2's environment.

Run:

```
pip list
```

Should identify:

```
tabulate  
rich
```

Run the application.

The Tabulate and Rich outputs should work.

Part 5 – Prove the Isolation

Demonstrate that the environments are independent.

Test

While Project 1's environment is active:

```
pip list
```

Check whether `rich` is available.

Then activate Project 2's environment and check again.

Similarly, check whether `Jinja2` and `PrettyTable` are available in Project 2.

Should understand that:

```
Project 1 .venv  
  ↓  
Jinja2  
PrettyTable  
  
Project 2 .venv  
  ↓  
Tabulate  
Rich
```

are isolated dependency environments.

Part 6 – Dependency Version Scenario

The trainer now introduces a new requirement.

The HR Report Generator has been tested with a specific version of PrettyTable.

The team decides that the project must continue using that version.

Project 1 should therefore specify:

```
prettytable==<specified-version>
```

Project 2 may use a different version of its dependencies.

Should understand why dependency versions should be explicitly recorded rather than relying on whatever version happens to be installed on the machine.

Part 7 – Recreate the Environment

To prove that `requirements.txt` is useful:

Step 1

Delete the Project 1 virtual environment.

```
rm -rf .venv
```

Step 2

Create it again:

```
python -m venv .venv
```

Step 3

Activate it.

Step 4

Install dependencies:

```
pip install -r requirements.txt
```

Step 5

Run the application.

The application should produce the same Jinja2 and PrettyTable output as before.

Repeat the same exercise for Project 2.

Final Expected Structure

```
python_training/
├── hr_report_generator/
│   ├── .venv/
│   ├── app.py
│   ├── requirements.txt
│   ├── templates/
│   │   └── employee_report.txt
│   └── employee_system/
│       ├── __init__.py
│       ├── employee.py
│       ├── salary.py
│       └── attendance.py
└── employee_cli/
    ├── .venv/
    ├── app.py
    ├── requirements.txt
    └── employee_system/
        ├── __init__.py
        ├── employee.py
        ├── salary.py
        └── attendance.py
```

Deliverables

Project 1 – HR Report Generator

Submit:

- Python modules
- `employee_system` package
- Jinja2 template
- Jinja2-generated employee report
- PrettyTable employee table
- `.venv`
- `requirements.txt`
- Sample output

Project 2 – Employee CLI

Submit:

- Python modules
- `employee_system` package
- Tabulate employee table
- Rich employee table
- `.venv`
- `requirements.txt`
- Sample output

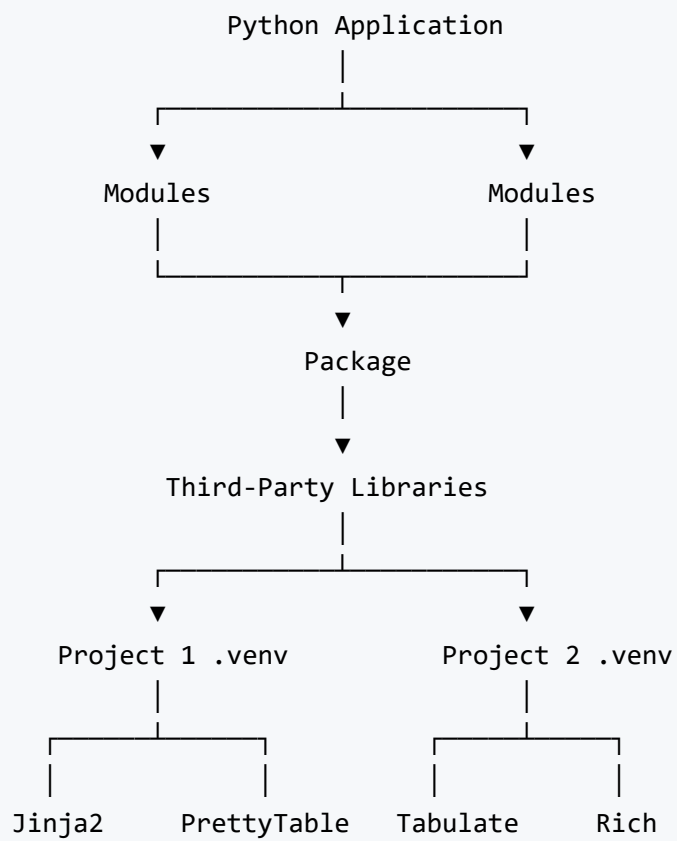
Documentation

Include a README explaining:

1. What is a module?
2. What is a package?
3. What is a virtual environment?
4. Why are two virtual environments used?
5. What is Jinja2 used for?
6. What is PrettyTable used for?
7. What is Tabulate used for?
8. What is Rich used for?
9. What is `requirements.txt` ?
10. Why should package versions be specified?

Final Learning Objective

Should understand the complete relationship:



Final Question

If both projects are on the same machine, why should they have separate virtual environments instead of installing all four libraries globally?